Cheat Sheet — Sesión 8: Cámaras y control 3D

Desarrollo de videojuegos 2D y 3D · Godot 4 / GDScript

Las funciones de la sesión

Función	Qué hace
`move_toward(actual, objetivo, paso)`	Mueve `actual` hacia `objetivo` como mucho `paso`, sin pasarse. Llamado cada cuadro: sube o baja de forma pareja.
`lerp(desde, hasta, t)`	Punto intermedio: t = 0 → el origen; t = 1 → el destino; t = 0.2 → 20 % del camino. Cada cuadro: acercamiento suave.
`clamp(v, min, max)` / `clampi`	Encierra un valor entre dos topes. `clampi` es para enteros (índices).
`a if condición else b`	Un `if/else` de una línea que devuelve un valor.
`i = (i + 1) % n`	Avanza un índice y vuelve a 0 después del último.

Rutas en el motor

- Correr la escena principal: **F5**. Correr la escena abierta: **F6**.
- Acción nueva: *Project Settings* → **Input Map** → *Add New Action* → **+ Add** → **+** → tecla → **OK**.
- Asignar un nodo a un `@export`: arrastrar el nodo desde el árbol hasta la propiedad en el Inspector.
- Cambiar la cámara que se ve: `camara.make_current()`.

Cámara de tercera persona (SpringArm3D)

Árbol: **Personaje → SpringArm3D → Camera3D**. Inspector del brazo: **Spring Length 4**, **Margin 0.2**, **Position Y 1.4**, **Rotation X −20** (el brazo lleva la cámara sobre su +Z: negativo la **sube**; con +20 queda a ras del suelo).

- **Borrar la Camera3D vieja** de la raíz (después de crear la nueva). Si no, se sigue viendo por ella.
- `top_level = true`: el brazo no gira con el personaje.
- `add_excluded_object(objetivo.get_rid())`: el brazo no choca con el propio personaje.
- Topes del giro vertical: `clamp(rotation.x, deg_to_rad(-55), deg_to_rad(25))`.

- Contra una pared el brazo se encoge (4.0 → 0.37 en la demo).
- **ESC** suelta el mouse.

Acelerar y frenar

```
var objetivo = direction * speed
var ritmo = aceleracion if direction.length() > 0.1 else friccion
velocity.x = move_toward(velocity.x, objetivo.x, ritmo * delta)
velocity.z = move_toward(velocity.z, objetivo.z, ritmo * delta)
```

Con `aceleracion` 12 llega a 3 m/s en 15 cuadros; con `friccion` 20 se detiene en 9.

Cámara isométrica que sigue

```
func _ready():
    top_level = true
    look_at_from_position(objetivo.global_position + distancia, objetivo.global
_position)

func _physics_process(delta):
    var destino = objetivo.global_position + distancia
    global_position = global_position.lerp(destino, suavizado * delta)
```

Se orienta **una vez** y después solo se traslada: así "adelante" no cambia. Con distancia (6, 8, 6), el giro horizontal es 45° y W lleva en diagonal.

La cámara de su juego

Juego	Cámara	distancia (o variante)
Desde arriba (salas, mapa, evacuación)	A1: la que sigue	(0, 12, 4): inclinada 71.6°
Isométrico, "2D diagonal"	A1	(6, 8, 6): giro 45°
De lado (2.5D)	A1, con Projection Orthogonal	(0, 2, 12); **Size** = metros de alto que se ven
Detrás del vehículo	A1 con suavizado bajo, o SpringArm3D hijo del kart	(0, 4, 9)
Primera persona	A2: en los ojos, sin brazo	`altura_ojos` 1.6; mouse a `rotation.y` y `rotation.x`, tope ±80°

- **Límites:** se limita el **foco** (lo que la cámara mira), no la cámara: `foco.x = clamp(foco.x, min, max)` y después `destino = foco + distancia`.
- **Apuntar:** `fov = lerp(fov, meta, rapidez * delta)`; con menos `fov`, zoom. La sensibilidad, por `fov ÷ fov_normal`.

Interactuar con lo cercano

```
for obj in get_tree().get_nodes_in_group("interactivos"):
    var d = distancia_a(obj)            # en el suelo: x y z
    if d <= alcance and (mejor == null or d < distancia_a(mejor)):
        mejor = obj
```

El grupo se pone en el panel **Node → Groups**. Mantener: `progreso += delta / tiempo_necesario`, completo con `>= 0.999`, y a 0 si se suelta.

Apuntar con un rayo

```
var consulta = PhysicsRayQueryParameters3D.create(desde, hacia)
consulta.exclude = [pj.get_rid()]       # que no se toque a sí mismo
var golpe = pj.get_world_3d().direct_space_state.intersect_ray(consulta)
if not golpe.is_empty():
    print(golpe.collider.name, golpe.position)
```

Desde	desde	hacia
El personaje (golpe, espada)	pecho: `pj.global_position + (0, 0.8, 0)`	`desde - pj.global_basis.z * alcance`
Centro de la cámara (FPS)	`cam.project_ray_origin(centro)`	`desde + cam.project_ray_normal(centro) * alcance`
El mouse (vista desde arriba)	igual, con `get_mouse_position()`	igual

El rayo solo ve **formas de colisión**. Daño: `if golpe.collider.has_method("recibir_golpe"): golpe.collider.recibir_golpe(danio)`; cadencia con `espera` que cuenta hacia atrás.

Extra: vehículo (kart de la demo)

La velocidad es **un número** (`velocidad_actual`); el "hacia dónde" lo pone la rotación.

```
var volante := Input.get_axis("move_right", "move_left")
var proporcion: float = abs(velocidad_actual) / velocidad_maxima
if proporcion > 0.05:                          # quieto no gira
    rotate_y(volante * velocidad_giro * proporcion * signf(velocidad_actual) *
delta)
var adelante := -transform.basis.z * velocidad_actual
velocity.x = adelante.x
velocity.z = adelante.z
```

SpringArm3D **hijo normal**, sin script ni `top_level`: gira con el kart.

Extra: runner por carriles

```
@export var carriles: Array[float] = [-3.0, 0.0, 3.0]
var indice_carril: int = 1

if Input.is_action_just_pressed("move_right"):
    indice_carril += 1
indice_carril = clampi(indice_carril, 0, carriles.size() - 1)
velocity.x = (carriles[indice_carril] - global_position.x) * velocidad_cambio
velocity.z = -velocidad_avance
```

Lejos va rápido, cerca va lento: se frena solo. Con `velocidad_cambio` 10, x = 3 × (1 − (5/6)ⁿ). Camera3D **hija directa**: el corredor nunca gira.

El repositorio del equipo (desde el Taller 7)

- **Uno por equipo**, público, creado desde la plantilla por una sola persona; los demás *Clone*. La URL va una vez en la hoja *Repositorio del juego* del Excel.
- **Su cuenta** en GitHub Desktop (*File* → *Options* → *Accounts*), en **su** PC: el autor del commit es quien se califica.
- **Cada tarea en su archivo**: A edita *personaje.gd*; B, el script de su nodo hijo; C, la escena y el script de su enemigo. En el Taller 8: la cámara en *camara_sigue.gd* o *camara_primera.gd*; *interactuar.gd* y *apuntar.gd* en nodos hijos. Nadie edita el archivo de otro.
- **Un commit por nivel**, con el mensaje exacto de la guía: `T7 N2 - Tarea A - arranque y frenado`. *Fetch* y *Pull* antes de empezar; **Push origin** después de cada commit.
- **El documento** lleva usuario de GitHub, URL del repo y el enlace a cada commit; el código no se copia.

Errores típicos

Síntoma	Causa
La cámara nueva no hace nada	Quedó la Camera3D vieja en la raíz
Cámara a ras del suelo	Rotation X del brazo en positivo; va en −20
Cámara pegada a la nuca	Falta `add_excluded_object`
La pantalla gira sola al caminar	Falta `top_level = true`
W lleva hacia un lado raro después de cambiar de cámara	No se actualizó `camara`
"Out of bounds get index"	Falta `%` (cámaras) o `clampi` (carriles)
El kart avanza hacia la cámara	`transform.basis.z` sin el menos
El kart gira quieto	Falta el `if proporcion > 0.05`
Un toque y se va al último carril	`is_action_pressed` en vez de `is_action_just_pressed`
La prueba no escribe nada	Corrieron con F5 en vez de F6, o la prueba está en otro nodo
La cámara que sigue no se mueve	El **Objetivo** está vacío, o falta `top_level`
En primera persona se ve el cuerpo	El Modelo sigue visible
"Nada a menos de..." siempre	El objeto no está en el grupo
El rayo le da al personaje a 0 m	Falta `exclude`
Pega en todos los cuadros	Falta `espera = cadencia`
El commit sale a nombre de otro	GitHub Desktop está con la cuenta de otro: *Options → Accounts*